AWS Deployment Quickstart

Direct path from local working code to AWS

Flipkart Clone Microservices Workspace

AWS Deployment Quickstart After Local Code Works

You already have the complete Flipkart Clone code in this workspace and the local Docker Compose setup. This document gives the direct path from local working code to AWS deployment.

Project path:

```
/home/user/flipkart-clone
```

Current Status

You should already have:

- Complete code for all 8 services
- Dockerfiles for every service
- Docker Compose local setup
- MongoDB local container
- React frontend
- API Gateway
- Product seed script
- AWS deployment documentation
- DevOps helper scripts
- GitHub Actions deployment workflow
- CodeBuild buildspec files
- VS Code AWS linking guide
- Monitoring guide

Pre-Flight Local Check

Run this before touching AWS:

```
cd /home/user/flipkart-clone
cp .env.example .env
docker compose up --build -d
docker compose ps
curl http://localhost:3000/health
curl "http://localhost:3000/api/products?limit=3"
```

If products are empty:

```
docker compose exec product-service npm run seed
```

Open:

```
http://localhost:8080
```

If this works, proceed to AWS.

Best AWS Deployment Path

For your project, the best practical path is:

```
Local code in VS Code
  ↓
Docker images built from each service
  ↓
Images pushed to Amazon ECR
  ↓
AWS infrastructure created with Console or Terraform
  ↓
ECS Fargate services run images
  ↓
ALB exposes frontend and /api gateway
  ↓
CloudWatch monitors everything
  ↓
GitHub Actions or CodePipeline automates future deployments
```

Recommended for learning:

1. Use AWS Console guide first.
2. Push images using `push-to-ecr.sh`.
3. Deploy ECS services manually once.
4. Then enable GitHub Actions or CodePipeline for future deployment.

Recommended for repeatable production:

1. Use Terraform to build infrastructure.
2. Use GitHub Actions or CodePipeline for CI/CD.
3. Use Secrets Manager for credentials.
4. Use CloudWatch alarms and dashboards.

Step 1: AWS Account and CLI

If you do not have AWS account:

1. Go to `https://aws.amazon.com/`.
2. Create account.
3. Add payment method.
4. Choose Basic Support.
5. Sign in to AWS Console.
6. Select region: `us-east-1`.

Install AWS CLI:

```
aws --version
```

Configure:

```
aws configure
```

Use:

```
Default region: us-east-1
Default output: json
```

Verify:

```
aws sts get-caller-identity
```

Step 2: Create ECR Repositories

You can create repositories with AWS Console or with the helper script.

Option A: AWS Console

Create these private repositories:

```
flipkart-api-gateway
flipkart-user-service
flipkart-product-service
flipkart-cart-service
flipkart-order-service
flipkart-payment-service
flipkart-notification-service
flipkart-frontend
```

Enable:

```
Scan on push: Enabled
Encryption: AES-256
```

Option B: Helper Script

```
cd /home/user/flipkart-clone
./scripts/aws-create-ecr-repositories.sh
```

Step 3: Push Docker Images to ECR

Get AWS account ID:

```
aws sts get-caller-identity --query Account --output text
```

Set it:

```
export AWS_ACCOUNT_ID=123456789012
export AWS_REGION=us-east-1
```

Build and push all images:

```
cd /home/user/flipkart-clone
./push-to-ecr.sh
```

This pushes:

```
latest
v1.0.0
```

The script automatically uses `Dockerfile.aws` for services that need Amazon DocumentDB TLS support.

Step 4: Create AWS Infrastructure

Use the existing PDF:

```
docs/AWS_CONSOLE_ONLY_SETUP_GUIDE.pdf
```

Follow these labs:

```
Lab 2: VPC and security groups
Lab 4: DocumentDB
Lab 5: Application Load Balancer
Lab 6: ECS cluster
Lab 7: ECS task definitions
Lab 8: Cloud Map service discovery
Lab 9: ECS services deployment
Lab 10: Auto scaling
Lab 11: CloudWatch monitoring
```

You can skip code creation because it already exists.

Step 5: Important AWS Runtime Values

Use these service discovery URLs in ECS task definitions:

```
USER_SERVICE_URL=http://user-service.flipkart.local:3001
PRODUCT_SERVICE_URL=http://product-service.flipkart.local:3002
CART_SERVICE_URL=http://cart-service.flipkart.local:3003
ORDER_SERVICE_URL=http://order-service.flipkart.local:3004
PAYMENT_SERVICE_URL=http://payment-service.flipkart.local:3005
NOTIFICATION_SERVICE_URL=http://notification-service.flipkart.local:3006
```

Use this frontend build value:

```
REACT_APP_API_URL=/api
```

The ALB must have a listener rule:

```
/api/* → flipkart-api-tg
Default → flipkart-frontend-tg
```

Step 6: DocumentDB URI

For AWS DocumentDB, use a URI like this:

```
mongodb://docdbadmin:URL_ENCODED_PASSWORD@CLUSTER_ENDPOINT:27017/flipkart-
users?tls=true&tlsCAFile=/app/global-
bundle.pem&replicaSet=rs0&readPreference=secondaryPreferred&retryWrites=false
```

Change database name for each service:

```
flipkart-users
flipkart-products
flipkart-carts
flipkart-orders
flipkart-payments
flipkart-notifications
```

Important:

- URL-encode special characters in password.
- `Dockerfile.aws` includes `/app/global-bundle.pem` for DocumentDB TLS.
- Keep `retryWrites=false` for DocumentDB.

Step 7: After ECS Services Are Created

Trigger deployment after pushing new image:

```
aws ecs update-service \
  --cluster flipkart-cluster \
  --service api-gateway-service \
  --force-new-deployment \
  --region us-east-1
```

Or deploy all services:

```
cp scripts/aws-env.example scripts/aws-env.local
# edit scripts/aws-env.local
./scripts/aws-ecs-force-deploy-template.sh
```

Step 8: Test AWS Deployment

Use ALB DNS:

```
curl http://ALB_DNS_NAME/api/health
curl http://ALB_DNS_NAME/api/products
```

Open browser:

```
http://ALB_DNS_NAME
```

Expected:

- Frontend loads.
- Register works.
- Login works.
- Products load.
- Cart works.
- Checkout creates order.

Step 9: Enable DevOps Pipeline

You have two ready options.

Option A: GitHub Actions

Manual all-service/selected-service workflow:

```
.github/workflows/deploy-to-aws-ecs.yml
```

Automatic per-service workflows:

```
.github/workflows/deploy-api-gateway.yml
.github/workflows/deploy-user-service.yml
.github/workflows/deploy-product-service.yml
.github/workflows/deploy-cart-service.yml
.github/workflows/deploy-order-service.yml
.github/workflows/deploy-payment-service.yml
.github/workflows/deploy-notification-service.yml
.github/workflows/deploy-frontend.yml
```

Add GitHub secrets:

```
AWS_ACCESS_KEY_ID
AWS_SECRET_ACCESS_KEY
AWS_REGION=us-east-1
ECS_CLUSTER=flipkart-cluster
REACT_APP_API_URL=/api
```

Then push to `main` or manually run workflow.

Option B: AWS CodePipeline and CodeBuild

Each service has a `buildspec.yml`, for example:

```
api-gateway/buildspec.yml
user-service/buildspec.yml
product-service/buildspec.yml
frontend/buildspec.yml
```

Use these when creating CodeBuild projects.

Step 10: Monitor

Use:

```
docs/MONITORING_AND_OPERATIONS_GUIDE.pdf
```

Minimum monitoring setup:

- CloudWatch log groups
- ECS CPU/memory alarms
- ALB 5XX alarm
- ALB unhealthy targets alarm

- DocumentDB CPU alarm
- AWS Budget alert

Final Download Checklist

Before downloading final workspace, confirm these files exist:

```
push-to-ecr.sh
scripts/aws-create-ecr-repositories.sh
scripts/aws-ecr-build-push-template.sh
scripts/aws-ecs-force-deploy-template.sh
.github/workflows/deploy-to-aws-ecs.yml
*/buildspec.yml
*/Dockerfile.aws for backend DB services
docs/AWS_CONSOLE_ONLY_SETUP_GUIDE.pdf
docs/VSCODE_AWS_INFRA_AND_BUILD_GUIDE.pdf
docs/MONITORING_AND_OPERATIONS_GUIDE.pdf
docs/REFERENCES.pdf
```

All are included in this workspace.